Master 2 TNAH, École nationale des chartes
4 novembre 2026

Marion Charpier



Introduction au langage de programmation Python

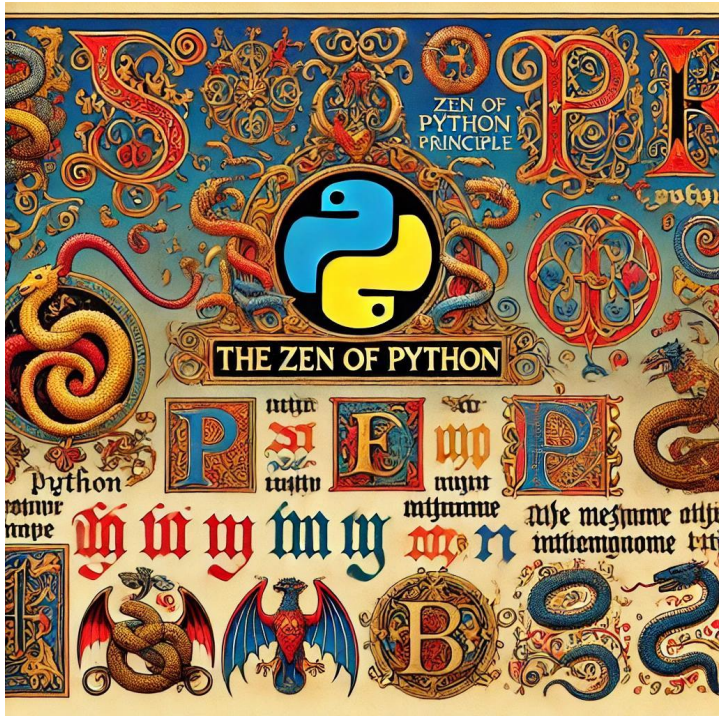
Séance 6 - Explorer les collections numérisées : IIIF

Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	14 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	4 novembre 2026	Explorer les collections numérisées : IIIF
7	10 novembre 2026	Du tableur à Python : CSV

Plan de la séance 6

1) **Correction de l'interrogation**
2) **IIIF : International Image Interoperability Framework**
   a) Qu'est-ce que IIIF ?
   b) Le manifeste et les annotations
   c) Les clés en @ : JSON-LD
   d) Deux versions du manifeste : v2 et v3
   e) Le canvas : une vue numérisée
   f) L'API Image : l'URL IIIF
3) **Télécharger des données avec requests**
   a) Envoyer une requête : get()
   b) Vérifier la réponse : status_code et raise_for_status()
   c) Télécharger une image : Content-Type
   d) Enregistrer une image : le mode "wb"
4) **Manipuler des images avec Pillow**
   a) PIL et la classe Image
   b) Les méthodes principales d'Image
5) **Notions à retenir**



Qu’est-ce que IIIF ?

Une communauté et un cadre

IIIF (*International Image Interoperability Framework*) désigne à la fois :

- une **communauté** : bibliothèques, musées, archives et universités du monde entier ;
- un **cadre d’interopérabilité** : des règles communes pour diffuser, présenter et annoter des images (et des documents audio ou vidéo) sur le Web.

Un objectif : décloisonner les collections

Le Physiologus de Berne (Burgerbibliothek, Cod. 318) est diffusé par e-codices, en Suisse. Le bestiaire Add MS 11283 est diffusé par la British Library, à Londres. Grâce à IIIF, on peut les afficher côte à côte dans le même visualiseur et les traiter avec le même code Python.

Un nom transparent

- **Framework** : un ensemble de conventions et d’outils sur lequel on construit des applications.
- **Interopérabilité** : la capacité de systèmes différents à échanger des informations et à travailler ensemble.

Deux API principales

Une **API** est un ensemble de règles qui permet à deux logiciels de communiquer.

- **L’API Présentation** décrit l’objet numérisé (sa structure, ses vues, ses métadonnées) dans un fichier JSON : le **manifeste**.
- **L’API Image** fournit les images elles-mêmes, à la taille et au cadrage voulus.

Un manifeste, plusieurs logiciels

Le même manifeste peut être ouvert dans un visualiseur, un outil d’annotation ou un programme Python. Chacun sait le lire, parce que tous suivent les mêmes règles.

Le manifeste et les annotations

Le manifeste : la notice d'un objet numérisé

Le manifeste est un fichier JSON qui rassemble tout ce qu'un logiciel doit savoir pour présenter un objet numérisé :

- son titre (label) et ses métadonnées (metadata) ;
- la liste ordonnée de ses vues, les **canvas** ;
- les liens vers l'image de chaque vue.

C'est l'unité de base de IIIF : pour partager un manuscrit, on partage l'URL de son manifeste.

Un format commun, des contenus différents

La structure est la même pour toutes les institutions, mais chacune choisit ses métadonnées. e-codices décrit Cod. 318 ainsi :

{"label": "Location", "value": "Bern"},
{"label": "Shelfmark", "value": "Cod. 318"},
{"label": "Number of Pages", "value": "131"}

Ces étiquettes sont propres à e-codices : un autre manifeste peut nommer la cote autrement, ou ne pas la donner.

Les annotations, à part du manifeste

Les transcriptions, les descriptions ou les commentaires d'une page sont des **annotations**. Elles sont souvent rangées dans des fichiers séparés, les **listes d'annotations**, qui ne contiennent aucune image. Le manifeste se contente d'y renvoyer par un lien.

Pourquoi les séparer ?

- Le manifeste reste léger et se charge plus vite.
- On peut corriger une transcription sans toucher au manifeste.
- Chacun ne charge que ce dont il a besoin.

Les clés en @ : JSON-LD

JSON-LD : du JSON pour les données liées

JSON-LD (*JSON for Linked Data*) est du JSON ordinaire, avec quelques conventions en plus. On le lit avec json.load(), exactement comme corpus.json.
Ces conventions servent aux **données liées** : chaque ressource reçoit une adresse unique sur le Web, ce qui permet de la relier à d'autres ressources, venues d'autres institutions.

Les mots-clés réservés

Les clés précédées de @ sont des mots-clés de JSON-LD :

- @context renvoie au document qui définit le vocabulaire du fichier, ici celui de l'API Présentation ;
- @id est l'identifiant unique de la ressource, sous forme d'URL ;
- @type indique la nature de la ressource : sc:Manifest pour un manifeste, sc:Canvas pour une vue.

Dans le manifeste de Berne

```
  "@context": "http://iiif.io/api/presentation/2/context.json",
  "@id":
https://www.e-codices.unifr.ch/metadata/iiif/bbb-0318/manifest.json",
  "@type": "sc:Manifest",
```

Trois sortes de clés

- @context, @id, @type : imposées par **JSON-LD** ;
- label, metadata, sequences : imposées par **IIIF**, sans @ ;
- les étiquettes à l'intérieur de metadata ("Location", "Shelfmark"…) : choisies par **l'institution**.

Un logiciel sait donc toujours où trouver le titre et les vues, même s'il ne connaît pas les métadonnées propres à e-codices.

Deux versions du manifeste : v2 et v3

L'API Présentation existe en deux versions, toutes deux encore en usage. e-codices publie le manifeste de Berne en **v2**, et la British Library publie celui d'Add MS 11283 en **v3**. Le contenu est le même, mais les clés changent.

Le même titre, deux façons de l'atteindre

```
manifeste_berne["label"]
# 'Bern, Burgerbibliothek, Cod. 318'
manifeste_londres["label"]
# {'en': ['…']}       un dictionnaire de langues
manifeste_londres["label"]["en"][0] # '…'
```

Rôle	v2 : Berne, Cod. 318	v3 : Londres, Add MS 11283
Vocabulaire	"@context": "…/presentation/2/context.json"	"@context": "…/presentation/3/context.json"
Identifiant	"@id" : "https://www.e-codices…bbb-0318/manifest.json"	"id" : "https://bl.digirati.io...vdc_100055965341.0x000001"
Type	"@type": "sc:Manifest"	"type": "Manifest"
Titre	"label": "Bern, Burgerbibliothek, Cod. 318"	"label": {"en": ["add ms 11283"]}
Métadonnées	{"label": "Shelfmark", "value": "Cod. 318"}	{"label": {"label": {"en": ["Title"]}, "value": {"en": ["Bestiary"]}}
Vues	["sequences"][0]["canvases"]	["items"]

Le canvas : une vue numérisée

Une vue, pas un folio

Un **canvas** est une vue numérisée : une page, mais aussi un plat de reliure, une garde, une mire de couleur ou une règle de mesure. Berne, Cod. 318 compte 131 feuillets, mais 281 canvas.

```
canvases = manifeste_berne["sequences"][0]["canvases"]
len(canvases)
# 281
canvases[0]["label"]
# 'Front cover'
canvases[4]["label"]
# '1r'
```

Ce que contient un canvas

- `label` : le nom de la vue (`"1r"`, `"Front cover"`) ;
- `width` et `height` : ses dimensions, en pixels ;
- l'image de la vue.

L'ordre de la liste est l'ordre de lecture : un visualiseur passe d'un canvas au suivant.

Descendre jusqu'à l'image

L'image n'est pas posée directement sur le canvas : elle y est rattachée par une **annotation**. Il faut donc descendre de plusieurs niveaux, et on y trouve deux adresses :

- **l'URL de l'image**, à télécharger telle quelle ;
- **la base de l'URL**, pour demander une autre région ou une autre taille (slide 11).

En v2 (Berne) :

```
image = canvases[4]["images"][0]["resource"]
image["@id"]
# '…/bbb-0318_001r.jp2/full/full/0/default/jpg'
image["service"]["@id"]
# '…/bbb-0318_001r.jp2'
```

En v3 (Add MS 11283) :

```
image = manifeste_londres["items"][9]["items"][0]["items"][0]["body"]
image["id"]
# '…/full/660,1024/0/default.jpg'
image["service"][1]["id"]
# '…/81055___vdc_100055965343.0x00000a'
```

L'API Image : l'URL IIIF

Deux modèles d'URL

Une requête de l'image elle-même (les pixels) :

{scheme}://{server}{/prefix}/{identifier}/{region}/{size}/{rotation}/{quality}.{format}

Une requête d'information sur l'image (du JSON) :

{scheme}://{server}{/prefix}/{identifier}/info.json

Les quatre premiers éléments forment la **base de l'URL** : on la trouve toute faite dans le canvas

Construire l'URL en Python

```
base = "https://..."    # base de l'URL (slide 10)
url = f"{base}/full/max/0/default.jpg"
# le f. 1r en entier
url_detail = f"{base}/1000,1500,2000,1500/800,/0/default.jpg"
# un détail, réduit à 800 pixels de large
url_info = f"{base}/info.json"
# les dimensions de l'image et les options du serveur
```

Paramètre	Exemples	Signification
région	full / x,y,l,h	l'image entière, ou un rectangle en pixels (x, y, largeur, hauteur)
taille	max (v3), full (v2) / 800,	la taille maximale, ou 800 pixels de large en gardant les proportions
rotation	0 / 90	en degrés, dans le sens des aiguilles d'une montre
qualité	default / gray	les couleurs d'origine, ou des niveaux de gris
format	jpg / png	le format du fichier

Envoyer une requête : get()

La bibliothèque requests

requests envoie des requêtes sur le Web depuis Python. Ce n'est pas un module natif : il faut l'installer (pip install requests), puis l'importer.

Télécharger le manifeste de Berne

```
import requests
url_manifeste = "https://…/manifest.json"   # le @id du manifeste
reponse = requests.get(url_manifeste, timeout=10)
```

- requests.get(url) demande la ressource au serveur et renvoie un objet **réponse**.
- timeout=10 : au-delà de 10 secondes sans réponse, on abandonne. Sans ce paramètre, le programme peut attendre indéfiniment.

Ce que contient la réponse

- reponse.status_code : le code de retour du serveur ;
- reponse.json() : le contenu, converti en dictionnaire Python ;
- reponse.content : le contenu brut, en *bytes* (pour une image) ;
- reponse.headers : les informations sur la réponse, comme le type du contenu.

```
manifeste_berne = reponse.json()
manifeste_berne["label"]
# 'Bern, Burgerbibliothek, Cod. 318'
```

C'est le même dictionnaire qu'en séance 5 avec json.load() : seule l'origine change, le Web au lieu d'un fichier.

Vérifier la réponse : status_code et raise_for_status()

Les codes de retour

Le serveur répond toujours par un code :

- 200 : OK ;
- 404 : ressource introuvable (une faute dans l'URL, par exemple) ;
- 500 : erreur du serveur.

Tester le code avec une condition

```
reponse = requests.get(url_manifeste, timeout=10)
if reponse.status_code == 200:
    manifeste_berne = reponse.json()
else:
    print(f"Échec du téléchargement : code {reponse.status_code}")
```

Lever une exception : raise_for_status()

raise_for_status() lève une exception (HTTPError) si le code indique une erreur (4xx ou 5xx). Il n'est alors plus nécessaire de tester status_code à la main. On gère l'exception avec try et except, comme en séance 4 :

```
try:
    reponse = requests.get(url_manifeste, timeout=10)
    reponse.raise_for_status()
    manifeste_berne = reponse.json()
except requests.exceptions.RequestException as erreur:
    print(f"Erreur lors du téléchargement : {erreur}")
```

RequestException couvre toutes les erreurs de requests : pas de réseau, délai dépassé, code d'erreur, contenu qui n'est pas du JSON.

Télécharger une image : Content-Type

Télécharger l'image du f. 1r

On reprend l'URL construite à la slide 11 :

```
url = f"{base}/full/max/0/default.jpg"
reponse = requests.get(url, timeout=10)
reponse.raise_for_status()
```

Une image n'est pas du JSON : on n'utilise pas reponse.json(), mais reponse.content, le contenu brut, en *bytes*.

Vérifier le type du contenu

L'en-tête Content-Type indique le type de la ressource reçue (son type MIME) :

- image/jpeg pour une image JPEG ;
- application/json ou application/ld+json pour un manifeste ;
- text/html; charset=UTF-8 pour une page web.

```
reponse.headers["Content-Type"]
# 'image/jpeg'
```

Le serveur ajoute parfois des précisions, comme ; charset=UTF-8. On teste donc avec in, pas avec == :

```
python
"image/jpeg" in reponse.headers["Content-Type"]
# True
```

Enregistrer une image : le mode "wb"

Écrire le fichier

```
from pathlib import Path

dossier = Path("images")
dossier.mkdir(exist_ok=True)
chemin = dossier / "add_ms_11283_f1r.jpg"

with open(chemin, "wb") as fichier:
    fichier.write(reponse.content)
```

- `mkdir(exist_ok=True)` crée le dossier, sans erreur s'il existe déjà.
- `dossier / "add_ms_11283_f1r.jpg"` construit le chemin, comme en séance 5.

Le mode "wb"

- "w" : écrire (*write*), comme en séance 5 ;
- "b" : en binaire (*binary*), pour des *bytes* et non du texte.

Sans le b, Python refuse :

`TypeError: write() argument must be str, not bytes`

PIL et la classe Image

PIL et Pillow

- **PIL** (*Python Imaging Library*) est une bibliothèque de traitement d'images, aujourd'hui abandonnée.
- **Pillow** en est la suite, activement maintenue : c'est elle qu'on utilise aujourd'hui.

Le piège : on **installe** pillow, mais on **importe** PIL.

```
# pip install pillow
from PIL import Image
```

Ce que fait Pillow

- Elle lit et écrit de nombreux formats : JPEG, PNG, TIFF, GIF…
- Elle transforme les images : recadrage, redimensionnement, rotation, conversion en niveaux de gris.

La classe Image

Image.open() ouvre un fichier et renvoie un **objet** image, comme Manuscrit("MS 24") renvoyait un objet en séance 3. Cet objet a des **attributs** et des **méthodes**.

```
chemin = Path("images") / "add_ms_11283_f1r.jpg"
image = Image.open(chemin)
```

```
image.size      # (largeur, hauteur) en pixels
image.format    # 'JPEG'
image.mode      # 'RGB' : une image en couleurs
```

Afficher l'image dans le notebook

On écrit simplement image en dernière ligne d'une cellule.

Méthodes principales d'Image

Transformer l'image

Chaque méthode renvoie une **nouvelle** image : l'original n'est pas modifié.

```
# Recadrer : (gauche, haut, droite, bas)
x, y, l, h = 1000, 1500, 2000, 1500
detail = image.crop((x, y, x + l, y + h))

# Redimensionner à 800 pixels de large, en gardant les proportions
largeur, hauteur = image.size
reduite = image.resize((800, round(hauteur * 800 / largeur)))

# Tourner de 90° dans le sens des aiguilles d'une montre
tournee = image.rotate(-90, expand=True)

# Enregistrer, ici au format PNG
detail.save(Path("images") / "add_ms_11283_f1r_detail.png")
```

Pillow et l'URL IIIF : les mêmes opérations, deux conventions

Pillow	URL IIIF	Attention
crop((gauche, haut, droite, bas))	region : x,y,l,h	Pillow attend des coordonnées de coins, IIIF une largeur et une hauteur.
resize((largeur, hauteur))	size : 800,	Pillow ne garde pas les proportions : il faut calculer la hauteur.
rotate(angle)	rotation : 90	Pillow tourne dans le sens inverse des aiguilles d'une montre, et rogne les coins sans expand=True

Avec l'URL IIIF, c'est le serveur qui transforme l'image ; avec Pillow, c'est Python, sur l'image enregistrée.

Notions à retenir

IIIF : un cadre d'interopérabilité

- **IIIF** : une communauté et des règles communes pour diffuser des images patrimoniales sur le Web.
- **Le manifeste** (API Présentation) : un fichier JSON qui décrit l'objet numérisé, avec son titre, ses métadonnées et ses vues.
- **Les clés en @** sont des mots-clés de JSON-LD. label, metadata et les vues sont imposés par IIIF ; le contenu de metadata est choisi par l'institution.
- **v2 et v3** : les clés changent (@id devient id, sequences devient items, et label devient un dictionnaire de langues). On lit @context pour savoir quelle version on a.
- **Le canvas** : une vue numérisée, pas un folio. L'image y est rattachée par une annotation.
- **L'API Image** : une URL qui décrit l'image voulue, avec region, size, rotation, quality et format.

requests : télécharger des données

- requests.get(url, timeout=10) envoie la requête.
- reponse.json() donne le contenu JSON, et reponse.content le contenu brut, en *bytes*.
- status_code et raise_for_status() vérifient la réponse, avec try / except.
- "image/jpeg" in reponse.headers["Content-Type"] vérifie le type du contenu.
- open(chemin, "wb") enregistre des *bytes*, dans un dossier créé avec pathlib.

Pillow : traiter des images

- On installe pillow, on importe PIL.
- Image.open() renvoie un objet image, qui a des attributs (size, mode) et des méthodes.
- crop(), resize(), rotate() et save() : les mêmes opérations que l'URL IIIF, avec d'autres conventions.